

**Санкт-Петербургский Национальный Исследовательский Университет
Информационных технологий, механики и оптики**

Отчет

Дисциплина: Web-программирование

Лабораторная работа 6.

Выполнил: Сарычев С.И.

Группа № K3321

Проверила: Марченко Е.В.

Санкт-Петербург

2024

Оглавление

Введение	3
Ход работы	4
1. Задание 1 – отсылка сообщения.....	4
2. Задание 2 – теорема Пифагора.....	6
3. Клиент для показа web-страниц	Ошибка! Закладка не определена.
Вывод	11

Введение

Цель работы: овладеть практическими навыками и умениями реализации web-серверов и использования сокетов.

Ход работы

1. Задание 1 – отсылка сообщения

В данном задании необходимо реализовать клиентскую и серверную часть приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента. Была использована библиотека socket. Код сервера представлен на рисунке 1, код клиента представлен на рисунке 2.

```
import socket

HOST = '127.0.0.1' # Локальный хост
PORT = 65432      # Порт (должен быть тот же, что и у клиента)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))      # Привязываем сокет к хосту и порту
    s.listen()               # Слушаем входящие подключения
    print('Сервер запущен, ожидаем подключение...')

    conn, addr = s.accept()   # Принимаем подключение
    with conn:
        print('Подключено к', addr)
        data = conn.recv(1024) # Получаем данные
        print('Получено сообщение от клиента:', data.decode())

        conn.sendall(b'Hello, client') # Отправляем ответ
```

Рисунок 1 – Код сервера задание 1

```
1 import socket
2
3 HOST = '127.0.0.1' # IP адрес сервера (в локальной сети или 127.0.0.1)
4 PORT = 65432      # Порт (должен совпадать с портом сервера)
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7     s.connect((HOST, PORT))          # Подключаемся к серверу
8     s.sendall(b'Hello, server')      # Отправляем сообщение
9     data = s.recv(1024)              # Получаем ответ
10    print('Ответ от сервера:', data.decode())
11
```

Рисунок 2 – Код клиента задание 1

Результат выполнения кода от лица сервера представлен на рисунке 3

```
bash: /Users/annafofomicheva/.profile: No such file or directory
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBook-Pro-Sergei:lab6 annafomicheva$ python3 task1/server.py
Сервер запущен, ожидаем подключение...
Подключено к ('127.0.0.1', 55310)
Получено сообщение от клиента: Hello, server
MacBook-Pro-Sergei:lab6 annafomicheva$ □
```

Рисунок 3 – Результат выполнения кода со стороны сервера в задании 1

2. Задание 2 – теорема Пифагора.

В данном задании необходимо реализовать клиентскую и серверную часть приложения. Клиент запрашивает у сервера выполнение математической операции, параметры, которые вводятся с клавиатуры. Сервер обрабатывает полученные данные и возвращает результат клиенту. Мне выпал вариант с теоремой Пифагора. По сути выполнение задание очень походе на первое, просто добавлены вычисления. Код сервера представлен на рисунке 4, код клиента представлен на рисунке 5.

```
4  HOST = '127.0.0.1'
5  PORT = 65432
6
7  def calculate_hypotenuse(a, b):
8      return math.sqrt(a**2 + b**2)
9
10 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
11     s.bind((HOST, PORT))
12     s.listen()
13     print('Сервер запущен, ожидаем подключение...')
14
15     conn, addr = s.accept()
16     with conn:
17         print('Подключено к', addr)
18         data = conn.recv(1024).decode()
19         print('Получено:', data)
20
21     try:
22         a_str, b_str = data.split(',')
23         a = float(a_str)
24         b = float(b_str)
25         c = calculate_hypotenuse(a, b)
26         result = f'Гипотенуза: {c:.2f}'
27     except Exception as e:
28         result = f'Ошибка вычислений: {e}'
29
30     conn.sendall(result.encode())
```

Рисунок 4 – Код сервера задание 2

```
task2 > client.py > ...
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 65432
5
6  # Ввод данных
7  a = input('Введите катет a: ')
8  b = input('Введите катет b: ')
9
10 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
11     s.connect((HOST, PORT))
12     message = f'{a},{b}'
13     s.sendall(message.encode())
14
15     data = s.recv(1024).decode()
16     print('Ответ от сервера:', data)
17
```

Рисунок 5 – Код клиента задание 2

Результат выполнения со стороны клиента представлен на рисунке 6.

```
For more details, please visit https://support.apple.com/kb/HT208050.
MacBook-Pro-Sergei:lab6 annafomicheva$ python3 task2/client.py
Введите катет a: 3
Введите катет b: 4
Ответ от сервера: Гипотенуза: 5.00
MacBook-Pro-Sergei:lab6 annafomicheva$
```

Рисунок 6 – результат выполнения со стороны клиента в задании 2

3. Задание 3 – html страница

В данном задании необходимо реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html. Код сервера представлен на рисунке 7.

```
4  HOST = '127.0.0.1'
5  PORT = 8080
6
7  with open('task3/index.html', 'r', encoding='utf-8') as file:
8      html_content = file.read()
9
10 # Преобразуем в байты заранее, чтобы корректно посчитать длину
11 html_bytes = html_content.encode('utf-8')
12
13 # Формируем корректный HTTP-ответ
14 http_response = b"".join([
15     b"HTTP/1.1 200 OK\r\n",
16     b"Content-Type: text/html; charset=utf-8\r\n",
17     f"Content-Length: {len(html_bytes)}\r\n".encode(),
18     b"\r\n", # Пустая строка – разделение заголовков и тела
19     html_bytes
20 ])
21
22 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
23     s.bind((HOST, PORT))
24     s.listen()
25     print(f"Сервер запущен на http://{HOST}:{PORT}")
26
27     while True:
28         conn, addr = s.accept()
29         with conn:
30             print(f"Клиент подключился: {addr}")
31             request = conn.recv(1024).decode()
32             print("Запрос клиента:")
33             print(request)
34
35             conn.sendall(http_response)
```

Рисунок 7 – Код сервера в задании 3

При переходе по адресу <http://127.0.0.1:8080> высвечивается следующая страница (Рисунок 8)

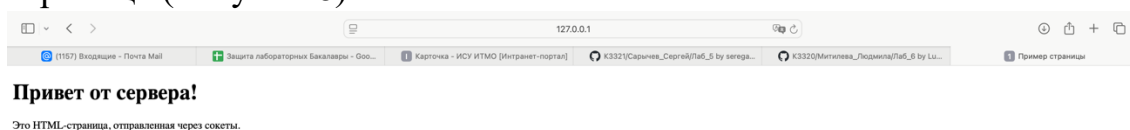


Рисунок 8 – HTML страница из задания 3

4. Задание 4 – многопользовательский чат.

В данном задании необходимо реализовать многопользовательский чат. Для этого использовалась библиотека websockets в python. Код серверной части представлен на рисунке 9.

```
1  import asyncio
2  import websockets
3
4  clients = {} # websocket: nickname
5
6  async def handler(websocket):
7      await websocket.send("Введите никнейм:")
8      nickname = await websocket.recv()
9
10     # Проверка на уникальность
11     if nickname in clients.values():
12         await websocket.send("Никнейм уже занят. Перезагрузите страницу.")
13         await websocket.close()
14         return
15
16     clients[websocket] = nickname
17     await broadcast(f"🟦 {nickname} присоединился к чату.")
18
19     try:
20         async for message in websocket:
21             await broadcast(f"{nickname}: {message}")
22     except:
23         pass
24     finally:
25         await broadcast(f"🔴 {nickname} покинул чат.")
26         del clients[websocket]
27
28  async def broadcast(message):
29      to_remove = []
30      for ws in clients:
31          try:
32              await ws.send(message)
33          except:
34              to_remove.append(ws)
35      for ws in to_remove:
36          del clients[ws]
37
38  # Создаём и запускаем цикл событий
39  async def main():
40      start_server = await websockets.serve(handler, "localhost", 8765)
41      print("Сервер запущен на ws://localhost:8765")
42      await start_server.wait_closed()
43
44  # Запускаем основной цикл
45  if __name__ == "__main__":
46      asyncio.run(main())
47
```

Рисунок 9 – Код сервера в задании 4

Далее была создана html страница ее код представлен на рисунке 10

```

1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4  |   <meta charset="UTF-8">
5  |   <title>Веб-чат</title>
6  |   <link rel="stylesheet" href="styles.css"> <!-- Подключаем CSS файл -->
7  </head>
8  <body>
9  |   <div class="container">
10 |     <h2>Чат</h2>
11 |     <div id="chat"></div>
12 |     <div>
13 |       <input id="msg" type="text" placeholder="Введите сообщение...">
14 |       <button onclick="sendMessage()">Отправить</button>
15 |     </div>
16 |   </div>
17 |
18 |   <!-- Подключаем внешний JavaScript файл -->
19 |   <script src="chat.js"></script>
20 </body>
21 </html>

```

Рисунок 10 – Код html страницы

Так же для корректной работы был создан javascript файл, его код представлен на рисунке 11

```

1  // chat.js
2
3  const ws = new WebSocket('ws://localhost:8765')
4  const chat = document.getElementById('chat')
5  const msgInput = document.getElementById('msg')
6
7  let nicknameSet = false
8
9  ws.onmessage = (event) => {
10 |   const msg = event.data
11 |
12 |   if (!nicknameSet && msg === 'Введите никнейм:') {
13 |     const nickname = prompt('Введите ваш никнейм:')
14 |     ws.send(nickname)
15 |     nicknameSet = true
16 |     return
17 |   }
18 |
19 |   if (msg.includes('Никнейм уже занят')) {
20 |     alert(msg)
21 |     ws.close()
22 |     return
23 |   }
24 |
25 |   chat.innerHTML += msg + '<br>'
26 |   chat.scrollTop = chat.scrollHeight
27 | }
28
29  function sendMessage() {
30 |   const text = msgInput.value
31 |   if (text !== '') {
32 |     ws.send(text)
33 |     msgInput.value = ''
34 |   }
35 | }
36
37  msgInput.addEventListener('keydown', (e) => {
38 |   if (e.key === 'Enter') sendMessage()
39 | })
40

```

Рисунок 11 – Javascript файл для подключения к серверу

Вывод

В ходе выполнения данной лабораторной работы были приобретены основы владения практическими навыками и умениями реализации web-серверов и использования сокетов.